

```

#include <windows.h>

#include <d3d11.h>

#include <d3dcompiler.h>

#pragma comment(lib, "d3d11.lib")
#pragma comment(lib, "d3dcompiler.lib")

// === НОВЫЙ КОД: ИДЕНТИФИКАТОРЫ ДЛЯ ВЕРХНЕГО МЕНЮ WINDOWS ===

#define IDM_FILE_OPEN_VIDEO 1001 // Открыть видеофайл (AVI/MP4)
#define IDM_FILE_OPEN_IMAGE 1002 // Открыть отдельное изображение (BMP/PNG)
#define IDM_FILE_SAVE_RESULT 1003 // Записать преобразованный файл
#define IDM_FILE_EXIT 1004 // Выход из программы

#define IDM_VIEW_SPLIT 1005 // Режим "До / После" (разделение экрана)
#define IDM_VIEW_RESULT_ONLY 1006 // Режим "Только результат"

#define IDM_HELP_ABOUT 1007 // О программе

// Структура вершины для нашего видеоэкрана
struct SimpleVertex
{
    float Pos[3]; // Координаты X, Y, Z в пространстве экрана
    float Tex[2]; // Текстурные координаты U, V для видеопотока
};

// Структура параметров кадра для передачи в шейдеры
struct ShaderConstants
{
    float width; // Ширина картинки
    float height; // Высота картинки
    float d_width; // Шаг одного пикселя по горизонтали (1.0 / width)
    float d_height; // Шаг одного пикселя по вертикали (1.0 / height)
}

```

```
};
```

```
// Глобальный указатель на буфер констант
```

```
ID3D11Buffer* g_pConstantBuffer = NULL;
```

```
// === НОВЫЙ КОД: ОБЪЕКТЫ ДЛЯ СВЕРТКИ АНАЛИЗА БЛОКОВ 4x4 ===
```

```
ID3D11RenderTargetView* g_pStage1RTV = NULL; // Холст для записи фиолетового анализа 3-го шейдера
```

```
ID3D11ShaderResourceView* g_pStage1SRV = NULL; // Ссылка на этот анализ для следующего шейдера
```

```
// === НОВЫЙ КОД: ОБЪЕКТЫ ДЛЯ УКРУПНЕННОГО АНАЛИЗА МАКРОБЛОКОВ 8x8 ===
```

```
ID3D11RenderTargetView* g_pStage2RTV = NULL; // Холст для записи макроанализа 4-го шейдера
```

```
ID3D11ShaderResourceView* g_pStage2SRV = NULL; // Ссылка на макроанализ для последующих шейдеров
```

```
// ОБЪЕКТЫ ДЛЯ ФИЛЬТРА ГЛОБАЛЬНОЙ КОНТРАСТНОСТИ (5-й ШЕЙДЕР)
```

```
ID3D11RenderTargetView* g_pStage3RTV = NULL; // Холст для записи HDR-фильтра
```

```
ID3D11ShaderResourceView* g_pStage3SRV = NULL; // Ссылка на этот результат
```

```
// === НОВЫЙ КОД: ПЕРЕМЕННАЯ ДЛЯ ХРАНЕНИЯ ПУТИ К ОТКРЫТОМУ ФАЙЛУ ===
```

```
wchar_t g_szSelectedFilePath[MAX_PATH] = L""; // Буфер для пути к картинке или видео
```

```
// --- ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ (СТРОГО ПО ОДНОМУ ЭКЗЕМПЛЯРУ): ---
```

```
ID3D11Device* g_pd3dDevice = NULL; // Видеокарта
```

```
ID3D11DeviceContext* g_pImmediateContext = NULL; // Контекст команд
```

```
IDXGISwapChain* g_pSwapChain = NULL; // Буфер экрана
```

```
ID3D11RenderTargetView* g_pRenderTargetView = NULL; // Окно вывода
```

```
ID3D11VertexShader* g_pVertexShader = NULL; // Вершинный шейдер
```

```
ID3D11PixelShader* g_pPixelShader = NULL; // Пиксельный шейдер
```

```
ID3D11PixelShader* g_pPixelShaderStage2 = NULL; // НОВОЕ: Указатель на 4-й шейдер анализа макроблоков 8x8
```

```
ID3D11PixelShader* g_pPixelShaderStage3 = NULL; // НОВОЕ: Указатель на 5-й шейдер адаптивной
фильтрации контраста
```

```
ID3D11PixelShader* g_pPixelShaderFinal = NULL; // НОВОЕ: Указатель на 6-й шейдер финальной
цветокоррекции и гаммы
```

```
ID3D11InputLayout* g_pVertexLayout = NULL; // Формат вершин
```

```
ID3D11Buffer* g_pVertexBuffer = NULL; // Буфер геометрии
```

```
ID3D11ShaderResourceView* g_pTextureSRV = NULL; // Наша текстура 2x2
```

```
ID3D11SamplerState* g_pSamplerState = NULL; // Жесткий сэмплер (Point)
```

```
LRESULT CALLBACK WindowProc(HWND hwnd, UINT uMsg, WPARAM wParam, LPARAM lParam);
```

```
void CleanupDevice(); // НОВОЕ: Объявляем функцию очистки заранее, чтобы убрать ошибку
компиляции
```

```
// Функция для компиляции HLSL-файлов
```

```
HRESULT CompileShaderFromFile(const WCHAR* szFileName, LPCSTR szEntryPoint, LPCSTR
szShaderModel, ID3DBlob** ppBlobOut)
```

```
{
```

```
    HRESULT hr = S_OK;
```

```
    ID3DBlob* pErrorBlob = NULL;
```

```
    hr = D3DCompileFromFile(szFileName, NULL, NULL, szEntryPoint, szShaderModel,
        D3DCOMPILE_ENABLE_STRICTNESS, 0, ppBlobOut, &pErrorBlob);
```

```
    if (FAILED(hr))
```

```
    {
```

```
        if (pErrorBlob)
```

```
        {
```

```
            OutputDebugStringA((char*)pErrorBlob->GetBufferPointer());
```

```
            pErrorBlob->Release();
```

```
        }
```

```
    return hr;
```

```

    }

    if (pErrorBlob) pErrorBlob->Release();

    return S_OK;
}

// Инициализация видеокарты и создание объектов
HRESULT InitDevice(HWND hwnd)
{
    HRESULT hr = S_OK;

    RECT rc;
    GetClientRect(hwnd, &rc);
    UINT width = rc.right - rc.left;
    UINT height = rc.bottom - rc.top;

    DXGI_SWAP_CHAIN_DESC sd = {};
    sd.BufferCount = 1;
    sd.BufferDesc.Width = width;
    sd.BufferDesc.Height = height;
    sd.BufferDesc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
    sd.BufferDesc.RefreshRate.Numerator = 60;
    sd.BufferDesc.RefreshRate.Denominator = 1;
    sd.BufferUsage = DXGI_USAGE_RENDER_TARGET_OUTPUT;
    sd.OutputWindow = hwnd;
    sd.SampleDesc.Count = 1;
    sd.SampleDesc.Quality = 0;
    sd.Windowed = TRUE;

    D3D_DRIVER_TYPE driverType = D3D_DRIVER_TYPE_HARDWARE;
    D3D_FEATURE_LEVEL featureLevel = D3D_FEATURE_LEVEL_11_0;

```

```

hr = D3D11CreateDeviceAndSwapChain(NULL, driverType, NULL, 0, &featureLevel, 1,
    D3D11_SDK_VERSION, &sd, &g_pSwapChain, &g_pd3dDevice, NULL, &g_pImmediateContext);
if (FAILED(hr)) return hr;

ID3D11Texture2D* pBackBuffer = NULL;
hr = g_pSwapChain->GetBuffer(0, __uuidof(ID3D11Texture2D), (LPVOID*)&pBackBuffer);
if (FAILED(hr)) return hr;

hr = g_pd3dDevice->CreateRenderTargetView(pBackBuffer, NULL, &g_pRenderTargetView);
pBackBuffer->Release();
if (FAILED(hr)) return hr;

g_pImmediateContext->OMSetRenderTargets(1, &g_pRenderTargetView, NULL);

D3D11_VIEWPORT vp;
vp.Width = (FLOAT)width;
vp.Height = (FLOAT)height;
vp.MinDepth = 0.0f;
vp.MaxDepth = 1.0f;
vp.TopLeftX = 0;
vp.TopLeftY = 0;
g_pImmediateContext->RSSetViewports(1, &vp);

// Компиляция Вершинного Шейдера
ID3DBlob* pVSBlob = NULL;
hr = CompileShaderFromFile(L"shaders.hlsl", "VS", "vs_4_0", &pVSBlob);
if (FAILED(hr)) return hr;

hr = g_pd3dDevice->CreateVertexShader(pVSBlob->GetBufferPointer(), pVSBlob->GetBufferSize(),
    NULL, &g_pVertexShader);
if (FAILED(hr))
{
    pVSBlob->Release();
}

```

```

    return hr;
}

// Описываем формат вершин для видеокарты (Input Layout)
D3D11_INPUT_ELEMENT_DESC layout[] =
{
    { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0, D3D11_INPUT_PER_VERTEX_DATA, 0 },
    { "TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT, 0, 12, D3D11_INPUT_PER_VERTEX_DATA, 0 },
};

UINT numElements = sizeof(layout) / sizeof(layout[0]);

hr = g_pd3dDevice->CreateInputLayout(layout, numElements, pVSBlob->GetBufferPointer(), pVSBlob->GetBufferSize(), &g_pVertexLayout);
pVSBlob->Release();
if (FAILED(hr)) return hr;

g_pImmediateContext->IASetInputLayout(g_pVertexLayout);

// Компиляция Пиксельного Шейдера
ID3DBlob* pPSBlob = NULL;
hr = CompileShaderFromFile(L"shaders.hlsl", "PS", "ps_4_0", &pPSBlob);
if (FAILED(hr)) return hr;

hr = g_pd3dDevice->CreatePixelShader(pPSBlob->GetBufferPointer(), pPSBlob->GetBufferSize(), NULL, &g_pPixelShader);
pPSBlob->Release();
if (FAILED(hr)) return hr;

// === НОВЫЙ КОД: КОМПИЛЯЦИЯ И СОЗДАНИЕ 4-ГО ШЕЙДЕРА (STAGE 2) ===
ID3DBlob* pPSStage2Blob = NULL;
// Указываем точку входа "PS_Stage2" — строго как имя функции в файле shaders.hlsl
hr = CompileShaderFromFile(L"shaders.hlsl", "PS_Stage2", "ps_4_0", &pPSStage2Blob);
if (FAILED(hr)) return hr;

```

```
hr = g_pd3dDevice->CreatePixelShader(pPSStage2Blob->GetBufferPointer(), pPSStage2Blob->GetBufferSize(), NULL, &g_pPixelShaderStage2);
```

```
pPSStage2Blob->Release(); // Освобождаем память временного буфера компиляции
```

```
if (FAILED(hr)) return hr;
```

```
// === КОНЕЦ НОВОГО КОДА ===
```

```
// === НОВЫЙ КОД: КОМПИЛЯЦИЯ И СОЗДАНИЕ 5-ГО ШЕЙДЕРА (STAGE 3) ===
```

```
ID3DBlob* pPSStage3Blob = NULL;
```

```
// Указываем точку входа "PS_Stage3" — строго как имя функции в файле shaders.hlsl
```

```
hr = CompileShaderFromFile(L"shaders.hlsl", "PS_Stage3", "ps_4_0", &pPSStage3Blob);
```

```
if (FAILED(hr)) return hr;
```

```
hr = g_pd3dDevice->CreatePixelShader(pPSStage3Blob->GetBufferPointer(), pPSStage3Blob->GetBufferSize(), NULL, &g_pPixelShaderStage3);
```

```
pPSStage3Blob->Release(); // Освобождаем память временного буфера компиляции
```

```
if (FAILED(hr)) return hr;
```

```
// === КОНЕЦ НОВОГО КОДА ===
```

```
// === НОВЫЙ КОД: КОМПИЛЯЦИЯ И СОЗДАНИЕ 6-ГО ШЕЙДЕРА (FINAL) ===
```

```
ID3DBlob* pPSFinalBlob = NULL;
```

```
// Указываем точку входа "PS_Final" — строго как имя функции в файле shaders.hlsl
```

```
hr = CompileShaderFromFile(L"shaders.hlsl", "PS_Final", "ps_4_0", &pPSFinalBlob);
```

```
if (FAILED(hr)) return hr;
```

```
hr = g_pd3dDevice->CreatePixelShader(pPSFinalBlob->GetBufferPointer(), pPSFinalBlob->GetBufferSize(), NULL, &g_pPixelShaderFinal);
```

```
pPSFinalBlob->Release(); // Освобождаем память временного буфера компиляции
```

```
if (FAILED(hr)) return hr;
```

```
// === КОНЕЦ НОВОГО КОДА ===
```

```
// Создаем тестовую текстуру 2x2 пикселя (Черный и Белый в шахматном порядке)
```

```
UINT textureData[] = {
```

```
0xFFFFFFFF, 0xFF000000,  
0xFF000000, 0xFFFFFFFF  
};
```

```
D3D11_TEXTURE2D_DESC desc = {};  
desc.Width = 2;  
desc.Height = 2;  
desc.MipLevels = 1;  
desc.ArraySize = 1;  
desc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;  
desc.SampleDesc.Count = 1;  
desc.Usage = D3D11_USAGE_DEFAULT;  
desc.BindFlags = D3D11_BIND_SHADER_RESOURCE;
```

```
D3D11_SUBRESOURCE_DATA initData = {};  
initData.pSysMem = textureData;  
initData.SysMemPitch = 2 * sizeof(UINT);
```

```
ID3D11Texture2D* pTexture = NULL;  
hr = g_pd3dDevice->CreateTexture2D(&desc, &initData, &pTexture);  
if (SUCCEEDED(hr))  
{  
    hr = g_pd3dDevice->CreateShaderResourceView(pTexture, NULL, &g_pTextureSRV);  
    pTexture->Release();  
}  
if (FAILED(hr)) return hr;
```

// Настраиваем режим POINT-фильтрации (строго без размытия)

```
D3D11_SAMPLER_DESC sampDesc = {};  
sampDesc.Filter = D3D11_FILTER_MIN_MAG_MIP_POINT;  
sampDesc.AddressU = D3D11_TEXTURE_ADDRESS_CLAMP;  
sampDesc.AddressV = D3D11_TEXTURE_ADDRESS_CLAMP;
```

```

sampDesc.AddressW = D3D11_TEXTURE_ADDRESS_CLAMP;
sampDesc.ComparisonFunc = D3D11_COMPARISON_NEVER;
sampDesc.MinLOD = 0;
sampDesc.MaxLOD = D3D11_FLOAT32_MAX;

hr = g_pd3dDevice->CreateSamplerState(&sampDesc, &g_pSamplerState);
if (FAILED(hr)) return hr;

// === НОВЫЙ КОД: СОЗДАЕМ БУФЕР КОНСТАНТ НА ВИДЕОКАРТЕ ===
D3D11_BUFFER_DESC cbd = {};
cbd.Usage = D3D11_USAGE_DEFAULT;
cbd.ByteWidth = sizeof(ShaderConstants); // Размер нашей структуры параметров
cbd.BindFlags = D3D11_BIND_CONSTANT_BUFFER; // Указываем видеокарте, что это буфер
констант
cbd.CPUAccessFlags = 0;

hr = g_pd3dDevice->CreateBuffer(&cbd, NULL, &g_pConstantBuffer);
if (FAILED(hr)) return hr;

// === КОНЕЦ НОВОГО КОДА ===

// === НОВЫЙ КОД: СОЗДАЕМ СКРЫТУЮ ТЕКСТУРУ ДЛЯ РЕЗУЛЬТАТОВ 3-ГО ШЕЙДЕРА ===
D3D11_TEXTURE2D_DESC stage1Desc = {};
stage1Desc.Width = 2; // Размер точно соответствует исходному кадру
stage1Desc.Height = 2;
stage1Desc.MipLevels = 1;
stage1Desc.ArraySize = 1;
stage1Desc.Format = DXGI_FORMAT_R8G8B8A8_UNORM; // Стандартный RGBA формат для
хранения данных
stage1Desc.SampleDesc.Count = 1;
stage1Desc.Usage = D3D11_USAGE_DEFAULT;

// Очень важные флаги: текстура будет одновременно холстом для записи и ресурсом для
чтения!

stage1Desc.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;

```

```

ID3D11Texture2D* pStage1Tex = NULL;

hr = g_pd3dDevice->CreateTexture2D(&stage1Desc, NULL, &pStage1Tex);

if (SUCCEEDED(hr))
{
    // Создаем "взгляд для записи" (RenderTargetView) — сюда 3-й шейдер будет писать данные
    g_pd3dDevice->CreateRenderTargetView(pStage1Tex, NULL, &g_pStage1RTV);

    // Создаем "взгляд для чтения" (ShaderResourceView) — отсюда 4-й шейдер будет их забирать
    g_pd3dDevice->CreateShaderResourceView(pStage1Tex, NULL, &g_pStage1SRV);

    pStage1Tex->Release(); // Саму текстуру видеокарта удержит внутри интерфейсов
}

if (FAILED(hr)) return hr;

// === КОНЕЦ НОВОГО КОДА ===

// === НОВЫЙ КОД: СОЗДАЕМ СКРЫТУЮ ТЕКСТУРУ ДЛЯ РЕЗУЛЬТАТОВ 4-ГО ШЕЙДЕРА ===
D3D11_TEXTURE2D_DESC stage2Desc = {};
stage2Desc.Width = 2;           // Размер по-прежнему соответствует тестовому кадру
stage2Desc.Height = 2;
stage2Desc.MipLevels = 1;
stage2Desc.ArraySize = 1;
stage2Desc.Format = DXGI_FORMAT_R8G8B8A8_UNORM; // Формат RGBA для хранения
макростатистики
stage2Desc.SampleDesc.Count = 1;
stage2Desc.Usage = D3D11_USAGE_DEFAULT;
stage2Desc.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;

ID3D11Texture2D* pStage2Tex = NULL;

hr = g_pd3dDevice->CreateTexture2D(&stage2Desc, NULL, &pStage2Tex);

if (SUCCEEDED(hr))
{
    // Создаем RenderTargetView — сюда 4-й шейдер запишет укрупненные данные 8x8

```

```

g_pd3dDevice->CreateRenderTargetView(pStage2Tex, NULL, &g_pStage2RTV);

// Создаем ShaderResourceView — отсюда данные заберет финальный сборочный шейдер
g_pd3dDevice->CreateShaderResourceView(pStage2Tex, NULL, &g_pStage2SRV);

pStage2Tex->Release();
}
if (FAILED(hr)) return hr;
// === КОНЕЦ НОВОГО КОДА ===

ID3D11Texture2D* pStage3Tex = NULL; // НОВОЕ: Объявляем указатель для третьей текстуры
// СОЗДАЕМ СКРЫТУЮ ТЕКСТУРУ ДЛЯ РЕЗУЛЬТАТОВ 5-ГО ШЕЙДЕРА
D3D11_TEXTURE2D_DESC stage3Desc = stage2Desc; // Копируем настройки формата и размеров
hr = g_pd3dDevice->CreateTexture2D(&stage3Desc, NULL, &pStage3Tex);

if (SUCCEEDED(hr))
{
    // Создаем RenderTargetView — сюда 5-й шейдер запишет результат фильтрации контраста
    hr = g_pd3dDevice->CreateRenderTargetView(pStage3Tex, NULL, &g_pStage3RTV);

    // Создаем ShaderResourceView — отсюда данные пойдут на финальный экран
    if (SUCCEEDED(hr))
    {
        hr = g_pd3dDevice->CreateShaderResourceView(pStage3Tex, NULL, &g_pStage3SRV);
    }

    pStage3Tex->Release(); // Передаем управление текстурой видеокарте
}
if (FAILED(hr)) return hr;

// Геометрия прямоугольного экрана
SimpleVertex vertices[] =

```

```

{
    {{ -1.0f, 1.0f, 0.0f }, { 0.0f, 0.0f } },
    {{ 1.0f, 1.0f, 0.0f }, { 1.0f, 0.0f } },
    {{ -1.0f, -1.0f, 0.0f }, { 0.0f, 1.0f } },
    {{ 1.0f, -1.0f, 0.0f }, { 1.0f, 1.0f } },
};

D3D11_BUFFER_DESC bd = {};
bd.Usage = D3D11_USAGE_DEFAULT;
bd.ByteWidth = sizeof(SimpleVertex) * 4;
bd.BindFlags = D3D11_BIND_VERTEX_BUFFER;
bd.CPUAccessFlags = 0;

D3D11_SUBRESOURCE_DATA InitData = {};
InitData.pSysMem = vertices;

hr = g_pd3dDevice->CreateBuffer(&bd, &InitData, &g_pVertexBuffer);
if (FAILED(hr)) return hr;

return S_OK;
}

// Обновленная функция отрисовки (Двухпроходный конвейер рендеринга)
// Обновленная функция отрисовки (Трехпроходный вычислительный конвейер)
// Обновленная функция отрисовки (Четырехпроходный вычислительный конвейер)
// Финальная функция отрисовки (Пятипроходный вычислительный конвейер)
void Render()
{
    if (g_pRenderTargetView == NULL) return;

    // 1. Общие настройки сетки экрана (прямоугольник из 4 вершин)
    UINT stride = sizeof(SimpleVertex);

```

```

UINT offset = 0;

g_pImmediateContext->IASetVertexBuffers(0, 1, &g_pVertexBuffer, &stride, &offset);

g_pImmediateContext->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLESTRIP);

g_pImmediateContext->IASetInputLayout(g_pVertexLayout);


// 2. Динамически запрашиваем у видеокарты физические размеры текущей активной текстуры
ShaderConstants cbData;

cbData.width = 2.0f; // Значения по умолчанию, если файл не выбран
cbData.height = 2.0f;


if (g_pTextureSRV)
{
    ID3D11Resource* pResource = NULL;
    g_pTextureSRV->GetResource(&pResource);
    if (pResource)
    {
        ID3D11Texture2D* pTex2D = (ID3D11Texture2D*)pResource;
        D3D11_TEXTURE2D_DESC desc;
        pTex2D->GetDesc(&desc);

        cbData.width = (float)desc.Width; // Подставляем реальную ширину открытого файла
        cbData.height = (float)desc.Height; // Подставляем реальную высоту открытого файла
        pResource->Release();
    }
}


// Рассчитываем точный шаг пикселя для ваших циклов анализа 4x4 и 8x8
cbData.d_width = 1.0f / cbData.width;
cbData.d_height = 1.0f / cbData.height;

g_pImmediateContext->UpdateSubresource(g_pConstantBuffer, 0, NULL, &cbData, 0, 0);

```

```

// Общие привязки буфера констант и Point-сэмплера к конвейеру GPU
g_pImmediateContext->VSSetShader(g_pVertexShader, NULL, 0);
g_pImmediateContext->PSSetConstantBuffers(0, 1, &g_pConstantBuffer);
g_pImmediateContext->PSSetSamplers(0, 1, &g_pSamplerState);

// Системный пустой указатель для сброса текстурных слотов (защита от конфликтов
чтения/записи)
ID3D11ShaderResourceView* nullSRV[] = { NULL };

// =====
// ПРОХОД 1: Скрытый анализ блоков 4x4 (3-й шейдер -> буфер Stage1)
// =====
float ClearColorBlack[] = { 0.0f, 0.0f, 0.0f, 1.0f };
g_pImmediateContext->ClearRenderTargetView(g_pStage1RTV, ClearColorBlack);

// Направляем вывод в первый скрытый холст Stage1
g_pImmediateContext->OMSetRenderTargets(1, &g_pStage1RTV, NULL);

// Включаем 3-й шейдер анализа блоков 4x4
g_pImmediateContext->PSSetShader(g_pPixelShader, NULL, 0);

// Подаем на вход стартовую шахматную текстуру кадра 2x2 в слот t0
g_pImmediateContext->PSSetShaderResources(0, 1, &g_pTextureSRV);

// Видеокарта рассчитывает первый этап статистики
g_pImmediateContext->Draw(4, 0);

// =====
// ПРОХОД 2: Скрытый макроанализ 8x8 (4-й шейдер -> буфер Stage2)
// =====
g_pImmediateContext->PSSetShaderResources(0, 1, nullSRV);

```

```
g_plmmediateContext->ClearRenderTargetView(g_pStage2RTV, ClearColorBlack);
```

```
// Переключаем вывод видеокарты на второй скрытый холст Stage2
```

```
g_plmmediateContext->OMSetRenderTargets(1, &g_pStage2RTV, NULL);
```

```
// Включаем 4-й шейдер укрупненного анализа макроблоков 8x8
```

```
g_plmmediateContext->PSSetShader(g_pPixelShaderStage2, NULL, 0);
```

```
// Подаем ему на вход результаты первого этапа из буфера Stage1 в слот t0
```

```
g_plmmediateContext->PSSetShaderResources(0, 1, &g_pStage1SRV);
```

```
// Видеокарта объединяет дисперсии независимых подвыборок
```

```
g_plmmediateContext->Draw(4, 0);
```

```
// =====
```

```
// ПРОХОД 3: Адаптивное сжатие контрастности (5-й шейдер -> буфер Stage3)
```

```
// =====
```

```
g_plmmediateContext->PSSetShaderResources(0, 1, nullSRV);
```

```
g_plmmediateContext->ClearRenderTargetView(g_pStage3RTV, ClearColorBlack);
```

```
// Переключаем вывод видеокарты на третий скрытый холст Stage3
```

```
g_plmmediateContext->OMSetRenderTargets(1, &g_pStage3RTV, NULL);
```

```
// Включаем 5-й шейдер фильтрации контрастности
```

```
g_plmmediateContext->PSSetShader(g_pPixelShaderStage3, NULL, 0);
```

```
// Подаем ему на вход укрупненную макростатистику из буфера Stage2 в слот t0
```

```
g_plmmediateContext->PSSetShaderResources(0, 1, &g_pStage2SRV);
```

```
// Видеокарта выполняет HDR-фильтрацию контраста
```

```
g_plmmediateContext->Draw(4, 0);
```

```

// =====
// ПРОХОД 4-5: Финальная цветокоррекция и вывод на реальный экран монитора
// =====
g_plmmediateContext->PSSetShaderResources(0, 1, nullSRV);

// Очищаем реальный экран монитора в мягкий серый цвет
float ClearColorGrey[] = { 0.75f, 0.75f, 0.75f, 1.0f };
g_plmmediateContext->ClearRenderTargetView(g_pRenderTargetView, ClearColorGrey);

// Возвращаем вывод видеокарты обратно на реальный экран монитора
g_plmmediateContext->OMSetRenderTargets(1, &g_pRenderTargetView, NULL);

// Включаем на прорисовку экрана наш новый 6-й шейдер (гамма-коррекция и финальный
штрих)
g_plmmediateContext->PSSetShader(g_pPixelShaderFinal, NULL, 0);

// Передаем на вход экрана отфильтрованные данные Stage3 в слот t0
g_plmmediateContext->PSSetShaderResources(0, 1, &g_pStage3SRV);

// Видеокарта выводит финальный результат на монитор
g_plmmediateContext->Draw(4, 0);

// Полный сброс текстурного слота для чистоты конвейера перед следующим кадром
g_plmmediateContext->PSSetShaderResources(0, 1, nullSRV);

// Выводим готовый кадр на экран монитора оператора
g_pSwapChain->Present(0, 0);
}

// Безопасное освобождение памяти при выходе
void CleanupDevice()

```

```

{
    if (g_pImmediateContext) g_pImmediateContext->ClearState();

    if (g_pStage1SRV) { g_pStage1SRV->Release(); g_pStage1SRV = NULL; }
    if (g_pStage1RTV) { g_pStage1RTV->Release(); g_pStage1RTV = NULL; }
    if (g_pStage2SRV) { g_pStage2SRV->Release(); g_pStage2SRV = NULL; } // НОВОЕ: Очистка ресурса
чтения Stage2
    if (g_pStage2RTV) { g_pStage2RTV->Release(); g_pStage2RTV = NULL; } // НОВОЕ: Очистка холста
записи Stage2
    if (g_pStage3SRV) { g_pStage3SRV->Release(); g_pStage3SRV = NULL; }
    if (g_pStage3RTV) { g_pStage3RTV->Release(); g_pStage3RTV = NULL; }
    if (g_pSamplerState) { g_pSamplerState->Release(); g_pSamplerState = NULL; }
    if (g_pConstantBuffer) { g_pConstantBuffer->Release(); g_pConstantBuffer = NULL; }
    if (g_pTextureSRV) { g_pTextureSRV->Release(); g_pTextureSRV = NULL; }
    if (g_pVertexBuffer) { g_pVertexBuffer->Release(); g_pVertexBuffer = NULL; }
    if (g_pVertexLayout) { g_pVertexLayout->Release(); g_pVertexLayout = NULL; }
    if (g_pPixelShader) { g_pPixelShader->Release(); g_pPixelShader = NULL; }
    if (g_pPixelShaderStage2) { g_pPixelShaderStage2->Release(); g_pPixelShaderStage2 = NULL; } //
НОВОЕ: Очистка 4-го шейдера анализа
    if (g_pPixelShaderStage3) { g_pPixelShaderStage3->Release(); g_pPixelShaderStage3 = NULL; } //
НОВОЕ: Очистка 5-го шейдера фильтрации
    if (g_pPixelShaderFinal) { g_pPixelShaderFinal->Release(); g_pPixelShaderFinal = NULL; } // НОВОЕ:
Очистка 6-го шейдера цветокоррекции
    if (g_pVertexShader) { g_pVertexShader->Release(); g_pVertexShader = NULL; }
    if (g_pRenderTargetView) { g_pRenderTargetView->Release(); g_pRenderTargetView = NULL; }
    if (g_pSwapChain) { g_pSwapChain->Release(); g_pSwapChain = NULL; }
    if (g_pImmediateContext) { g_pImmediateContext->Release(); g_pImmediateContext = NULL; }
    if (g_pd3dDevice) { g_pd3dDevice->Release(); g_pd3dDevice = NULL; }
}

```

// === НОВЫЙ КОД: ФУНКЦИЯ СОЗДАНИЯ СИСТЕМНОГО МЕНЮ ===

```
void CreateAppMenu(HWND hwnd)
```

```

{
    HMENU hMenu = CreateMenu();    // Главная горизонтальная полоса

```

```

HMENU hFileMenu = CreatePopupMenu(); // Выпадающее меню "Файл"
HMENU hViewMenu = CreatePopupMenu(); // Выпадающее меню "Вид"
HMENU hHelpMenu = CreatePopupMenu(); // Выпадающее меню "Справка"

// Заполняем меню "Файл"
AppendMenuW(hFileMenu, MF_STRING, IDM_FILE_OPEN_VIDEO, L"Открыть видео поток
(AVI/MP4)...");

AppendMenuW(hFileMenu, MF_STRING, IDM_FILE_OPEN_IMAGE, L"Открыть кадр/картинку
(BMP/PNG)...");

AppendMenuW(hFileMenu, MF_SEPARATOR, 0, NULL); // Разделительная линия
AppendMenuW(hFileMenu, MF_STRING, IDM_FILE_SAVE_RESULT, L"Записать преобразованный
файл...");

AppendMenuW(hFileMenu, MF_SEPARATOR, 0, NULL);
AppendMenuW(hFileMenu, MF_STRING, IDM_FILE_EXIT, L"Выход");

// Заполняем меню "Вид"
AppendMenuW(hViewMenu, MF_STRING | MF_CHECKED, IDM_VIEW_RESULT_ONLY, L"Только
обработанный результат");

AppendMenuW(hViewMenu, MF_STRING, IDM_VIEW_SPLIT, L"Режим \"До / После\" (Разделение
экрана)");

// Заполняем меню "Справка"
AppendMenuW(hHelpMenu, MF_STRING, IDM_HELP_ABOUT, L"О программе...");

// Прикрепляем выпадающие списки к главной полосе меню
AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hFileMenu, L"Файл");
AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hViewMenu, L"Вид");
AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hHelpMenu, L"Справка");

// Физически вешаем меню на наше окно программы
SetMenu(hwnd, hMenu);
}

// === КОНЕЦ НОВОГО КОДА ===

```

```

#include <wincodec.h> // Подключаем заголовки системного декодера WIC

// === НОВЫЙ КОД: БЕЗОПАСНАЯ ЗАГРУЗКА ЛЮБЫХ КАРТИНОК ЧЕРЕЗ WIC НА ВИДЕОКАРТУ ===
HRESULT LoadTextureFromFile(const WCHAR* szFileName)
{
    HRESULT hr = S_OK;

    // 1. Создаем фабрику декодеров WIC
    IWICImagingFactory* pWICFactory = NULL;

    hr = CoCreateInstance(CLSID_WICImagingFactory, NULL, CLSCTX_INPROC_SERVER,
        __uuidof(IWICImagingFactory), (LPVOID*)&pWICFactory);
    if (FAILED(hr)) return hr;

    // 2. Открываем файл картинки на диске
    IWICBitmapDecoder* pDecoder = NULL;

    hr = pWICFactory->CreateDecoderFromFilename(szFileName, NULL, GENERIC_READ,
        WICDecodeMetadataCacheOnDemand, &pDecoder);
    if (FAILED(hr)) { pWICFactory->Release(); return hr; }

    // 3. Берем самый первый кадр из файла (для BMP/PNG он один)
    IWICBitmapFrameDecode* pFrame = NULL;

    hr = pDecoder->GetFrame(0, &pFrame);
    if (FAILED(hr)) { pDecoder->Release(); pWICFactory->Release(); return hr; }

    // 4. Принудительно конвертируем пиксели в стандартный формат Direct3D 11 (RGBA 32-бит)
    IWICFormatConverter* pConverter = NULL;

    hr = pWICFactory->CreateFormatConverter(&pConverter);
    if (FAILED(hr)) { pFrame->Release(); pDecoder->Release(); pWICFactory->Release(); return hr; }

    hr = pConverter->Initialize(pFrame, GUID_WICPixelFormat32bppPBGRA, WICBitmapDitherTypeNone,
        NULL, 0.0f, WICBitmapPaletteTypeCustom);

    if (FAILED(hr)) { pConverter->Release(); pFrame->Release(); pDecoder->Release(); pWICFactory-
        >Release(); return hr; }
}

```

```

// 5. Узнаем физические размеры загруженной картинки
UINT imgWidth = 0, imgHeight = 0;
pConverter->GetSize(&imgWidth, &imgHeight);

// Выделяем временный буфер в оперативной памяти компьютера под пиксели
UINT* pPixelsBuffer = new UINT[imgWidth * imgHeight];

hr = pConverter->CopyPixels(NULL, imgWidth * sizeof(UINT), imgWidth * imgHeight * sizeof(UINT),
(BYTE*)pPixelsBuffer);

if (SUCCEEDED(hr))
{
    // 6. Если старая текстура уже была в памяти — чисто освобождаем её ресурсы перед
    перезаписью!

    if (g_pTextureSRV) { g_pTextureSRV->Release(); g_pTextureSRV = NULL; }

    // 7. Создаем новую текстуру прямо в видеопамяти под размеры нашей картинки
    D3D11_TEXTURE2D_DESC desc = {};

    desc.Width = imgWidth;
    desc.Height = imgHeight;
    desc.MipLevels = 1;
    desc.ArraySize = 1;

    desc.Format = DXGI_FORMAT_R8G8B8A8_UNORM; // Тот самый формат, под который настроен
    3-й шейдер

    desc.SampleDesc.Count = 1;
    desc.Usage = D3D11_USAGE_DEFAULT;
    desc.BindFlags = D3D11_BIND_SHADER_RESOURCE;

    D3D11_SUBRESOURCE_DATA initData = {};
    initData.pSysMem = pPixelsBuffer;
    initData.SysMemPitch = imgWidth * sizeof(UINT);

    ID3D11Texture2D* pTexture = NULL;
    hr = g_pd3dDevice->CreateTexture2D(&desc, &initData, &pTexture);

```

```

    if (SUCCEEDED(hr))
    {
        // Создаем ресурс чтения для шейдеров плеера
        hr = g_pd3dDevice->CreateShaderResourceView(pTexture, NULL, &g_pTextureSRV);
        pTexture->Release();
    }
}

// === ЗОЛОТОЕ ПРАВИЛО: СТРОЖАЙШЕ И ЧИСТО ЗАКРЫВАЕМ ВСЕ РЕСУРСЫ И ПАМЯТЬ! ===
delete[] pPixelsBuffer; // Удаляем временный массив пикселей из ОЗУ
pConverter->Release(); // Закрываем конвертер WIC
pFrame->Release(); // Освобождаем кадр
pDecoder->Release(); // Закрываем файл на диске намертво!
pWICFactory->Release(); // Уничтожаем фабрику декодеров

return hr;
}

// === НОВЫЙ КОД: ФУНКЦИЯ ВЫЗОВА СИСТЕМНОГО ПРОВОДНИКА WINDOWS ===
bool OpenFileDialog(HWND hwnd, bool bOpenVideo)
{
    OPENFILENAMEW ofn = {};
    wchar_t szFile[MAX_PATH] = L"";

    ofn.lStructSize = sizeof(ofn);
    ofn.hwndOwner = hwnd;
    ofn.lpstrFile = szFile;
    ofn.nMaxFile = sizeof(szFile) / sizeof(szFile[0]);

    if (bOpenVideo)
    {
        // Фильтр файлов для видеопотока
    }
}

```

```

    ofn.lpstrFilter = L"Видео файлы (AVI, MP4)\0*.avi;*.mp4\0Все файлы (*.*)\0*.*\0";
    ofn.lpstrTitle = L"Выберите видео поток для дефектоскопа";
}

else
{
    // Фильтр файлов для отдельных изображений
    ofn.lpstrFilter = L"Изображения (BMP, PNG)\0*.bmp;*.png\0Все файлы (*.*)\0*.*\0";
    ofn.lpstrTitle = L"Выберите кадр/картинку для анализа";
}

ofn.Flags = OFN_PATHMUSTEXIST | OFN_FILEMUSTEXIST;

// Распахиваем системное окно Проводника
if (GetOpenFileNameW(&ofn))
{
    // Если пользователь выбрал файл и нажал "Открыть" — копируем путь в нашу глобальную
    переменную
    wcscpy_s(g_szSelectedFilePath, MAX_PATH, ofn.lpstrFile);
    return true;
}

return false; // Если пользователь нажал "Отмена"
}

// === КОНЕЦ НОВОГО КОДА ===

// Главная точка входа Windows-приложения
int WINAPI wWinMain(_In_ HINSTANCE hInstance, _In_opt_ HINSTANCE hPrevInstance, _In_ PWSTR
pCmdLine, _In_ int nCmdShow)
{
    // === НОВЫЙ КОД: ИНИЦИАЛИЗАЦИЯ СИСТЕМНОЙ БИБЛИОТЕКИ COM ДЛЯ ДВИЖКА WIC ===
    (void)CoInitializeEx(NULL, COINIT_APARTMENTTHREADED); // Запускает поддержку WIC в Windows

    const wchar_t CLASS_NAME[] = L"MyVideoPlayerWindowClass";

```

```

WNDCLASS wc = { };

wc.lpfnWndProc = WindowProc;

wc.hInstance = hInstance;

wc.lpszClassName = CLASS_NAME;

wc.hCursor = LoadCursor(NULL, IDC_ARROW);

RegisterClass(&wc);


HWND hwnd = CreateWindowEx(0, CLASS_NAME, L"Мой Шейдерный Видеоплеер",
    WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, CW_USEDEFAULT, 800, 600, NULL, NULL, hInstance,
    NULL);


if (hwnd == NULL) return 0;

ShowWindow(hwnd, nCmdShow);


CreateAppMenu(hwnd); // НОБОЕ: Физически включаем меню на экране сразу после показа окна


if (FAILED(InitDevice(hwnd)))
{
    CleanupDevice();

    CoUninitialize(); // НОБОЕ: Чисто закрываем и освобождаем ресурсы COM-потока перед
    выходом!

    return 0;
}


MSG msg = { };

while (WM_QUIT != msg.message)
{
    if (PeekMessage(&msg, NULL, 0, 0, PM_REMOVE))
    {
        TranslateMessage(&msg);

        DispatchMessage(&msg);
    }

    else

```

```

    {
        Render();
    }
}

CleanupDevice();

CoUninitialize(); // НОВОЕ: Чисто закрываем и освобождаем ресурсы COM-потока перед
выходом!

return 0;
}

// Главная точка входа Windows-приложения
LRESULT CALLBACK WindowProc(HWND hwnd, UINT uMsg, WPARAM wParam, LPARAM lParam)
{
    switch (uMsg)
    {
        // === НОВЫЙ КОД: ОБРАБОТКА НАЖАТИЙ НА ПУНКТЫ МЕНЮ ===
        case WM_COMMAND:
            // В wParam система Windows передает ID нажатого пункта меню
            switch (LOWORD(wParam))
            {
                case IDM_FILE_EXIT:
                    // Если нажали "Выход" — закрываем окно программы
                    DestroyWindow(hwnd);
                    return 0;

                case IDM_HELP_ABOUT:
                    // Если нажали "О программе" — показываем красивое информационное окно
                    MessageBoxW(hwnd,
                        L"Шейдерный Видеоплеер-Дефектоскоп v2026\n\n"
                        L"Реализован пятипроходный вычислительный конвейер.\n"
                        L"Алгоритмы фильтрации локального контраста и дисперсии макроблоков 4x4 и 8x8.\n\n"

```

```

        L"Разработчик математического ядра: Курчанов А. Ф.",
        L"О программе",
        MB_OK | MB_ICONINFORMATION);
return 0;

case IDM_FILE_OPEN_VIDEO:
    // Вызываем Проводник в режиме фильтрации видео (передаем true)
    if (OpenFileDialog(hwnd, true))
    {
        // Если файл выбран успешно — выведем его путь на экран для проверки
        MessageBoxW(hwnd, g_szSelectedFilePath, L"Видео поток успешно подключен", MB_OK |
        MB_ICONINFORMATION);

        // TODO: Здесь на следующем шаге мы запустим инициализацию Media Foundation
    }
    return 0;

case IDM_FILE_OPEN_IMAGE:
    // 1. Распахиваем Проводник для выбора картинки
    if (OpenFileDialog(hwnd, false))
    {
        // 2. Если файл выбран — загружаем его пиксели в Point-текстуру через WIC
        if (SUCCEEDED(LoadTextureFromFile(g_szSelectedFilePath)))
        {
            // 3. Вызываем принудительную перерисовку окна, чтобы видеокарта
            // мгновенно прогнала новую картинку через все 5 проходов шейдеров!
            InvalidateRect(hwnd, NULL, FALSE);
        }
        else
        {
            MessageBoxW(hwnd, L"Не удалось декодировать файл изображения через WIC.",
            L"Ошибка дефектоскопа", MB_OK | MB_ICONERROR);
        }
    }

```

```

    }

    return 0;

case IDM_FILE_SAVE_RESULT:

case IDM_VIEW_SPLIT:

case IDM_VIEW_RESULT_ONLY:

    // Для остальных пунктов пока выведем временную заглушку, чтобы видеть отклик
    MessageBoxW(hwnd, L"Этот блок конвейера находится в режиме подключения данных.",
L"Информация", MB_OK | MB_ICONINFORMATION);

    return 0;

}

break;

// === КОНЕЦ НОВОГО КОДА ===

case WM_SIZE:

    if (g_pSwapChain)
    {
        if (g_pRenderTargetView) { g_pRenderTargetView->Release(); g_pRenderTargetView = NULL; }

        UINT width = LOWORD(IParam);

        UINT height = HIWORD(IParam);

        g_pSwapChain->ResizeBuffers(0, width, height, DXGI_FORMAT_UNKNOWN, 0);

        ID3D11Texture2D* pBackBuffer = NULL;

        g_pSwapChain->GetBuffer(0, __uuidof(ID3D11Texture2D), (LPVOID*)&pBackBuffer);

        if (pBackBuffer)
        {
            g_pd3dDevice->CreateRenderTargetView(pBackBuffer, NULL, &g_pRenderTargetView);

            pBackBuffer->Release();
        }

        g_pImmediateContext->OMSetRenderTargets(1, &g_pRenderTargetView, NULL);

        D3D11_VIEWPORT vp;

```

```
    vp.Width = (FLOAT)width;
    vp.Height = (FLOAT)height;
    vp.MinDepth = 0.0f;
    vp.MaxDepth = 1.0f;
    vp.TopLeftX = 0;
    vp.TopLeftY = 0;
    g_pImmediateContext->RSSetViewports(1, &vp);
}

return 0;

case WM_DESTROY:
    PostQuitMessage(0);
    return 0;
}

return DefWindowProc(hwnd, uMsg, wParam, lParam);
}
```